

CHAPITRE 1 : PL/SQL -Généralités

1. Définition et Objectifs

- PL/SQL : Langage procédural Oracle étendant SQL avec des structures de programmation (boucles, conditions, exceptions).
- Différence avec SQL :
 - SQL : Langage déclaratif normé (ANSI/ISO).
 - PL/SQL : Langage procédural propriétaire (Oracle).
- Utilisations principales :
 - Procédures stockées, déclencheurs (*triggers*).
 - Traitement structuré des données (ex : validation, logique métier).

2. Structure de Base d'un Bloc PL/SQL

```
sql Copy Download  
  
[DECLARE  
  -- Variables (optionnel)]  
BEGIN  
  -- Instructions SQL + logique (obligatoire)  
[EXCEPTION  
  -- Gestion d'erreurs (optionnel)]  
END;
```

3. Avantages Clés

- **Performance :**
 - Envoi de **blocs complets** au serveur → moins de requêtes réseau.
 - Code **compilé et stocké** en mémoire cache.
- **Intégration SQL/PLSQL :**
 - Compatibilité des types de données (ex : VARCHAR2, DATE).
 - Clause INTO pour stocker les résultats SQL dans des variables.
- **Modularité :**
 - Réutilisable via procédures/fonctions stockées.

4. Procédures Stockées

- **Définition :** Code PL/SQL stocké sur le serveur et appelé via CALL.
- **Avantages :**
 - **Réduction du trafic réseau** (exécution côté serveur).
 - **Sécurité :** Masquage de la structure de la BD.
 - **Maintenance centralisée** (mise à jour unique).

5. Architecture

- **Moteurs :**
 - **PL/SQL :** Gère la logique procédurale.
 - **SQL :** Exécute les requêtes.
- **Processus :**
 1. Appel d'une procédure → chargement en mémoire.
 2. Exécution coordonnée par les moteurs.

6. Normes et Alternatives

- **SQL/PSM :** Norme pour les modules stockés.
- **Équivalents :**
 - T-SQL (SQL Server), PL/PgSQL (PostgreSQL).

Chapitre 2 – Les types de données et les variables

Les quatre catégories de données :

- **Scalaire** : types atomiques, utilisés pour définir une colonne d'une table.
- **Composés** : types contenant plusieurs composants (ex. RECORD).
- **Références** : types REF, permettent de référencer d'autres objets.
- **Grands objets (LOB)** : stockent jusqu'à 4 Go de données binaires ou texte.

Types de données scalaires

Types numériques

- **NUMBER(P,S)** : P = précision (chiffres significatifs), S = échelle (décimales). Sous-types : INTEGER, DECIMAL, FLOAT...
- **BINARY_INTEGER** : entiers signés entre -2 147 483 647 et +2 147 483 647. Sous-types : NATURAL, POSITIVE, NATURALN, POSITIVEN, SIGNTYPE.
- **PLS_INTEGER** : plus rapide que NUMBER et BINARY_INTEGER, génère une exception en cas de dépassement.
- **BINARY_FLOAT / BINARY_DOUBLE** : nombres à virgule flottante (Oracle 10g+), format IEEE-754, utilisés pour les calculs scientifiques.

Types de caractères

- **CHAR(size)** : chaînes de longueur fixe, taille exprimée en bytes ou CHAR. Sous-type : CHARACTER.
- **VARCHAR2(size)** : chaînes de longueur variable, max 32767 bytes (attention à la limite de stockage en table : 4000).
- **LONG et LONG RAW** : similaires à VARCHAR2 et RAW, mais obsolètes. MAX : 32760 bytes.
- **RAW(size)** : données binaires non interprétées, jusqu'à 32767 bytes. En colonne : RAW = 2000 bytes, LONG RAW = 2 Go.
- **ROWID / UROWID** : identifiant de ligne (adresse mémoire). ROWID = physique, UROWID = logique ou externe. Fonctions : CAST, ROWIDTOCHAR, CHARTOROWID.
- **NCHAR / NVARCHAR2** : similaires à CHAR et VARCHAR2 mais basés sur le jeu de caractères national.

Type booléen

- **BOOLEAN** : valeurs TRUE, FALSE, NULL. Utilisable uniquement en PL/SQL (pas en SQL standard).

Types DATE, TIME, INTERVAL

- **DATE** : inclut la date et l'heure (jusqu'aux secondes).
- **TIMESTAMP(n)** : identique à DATE avec précision sur les fractions de seconde.
- **INTERVAL DAY(p) TO SECOND(p)** : intervalle jour-heure-minute-seconde.
- **INTERVAL YEAR(p) TO MONTH** : intervalle année-mois.

Types de données LOBS

- Les types LOB (Large OBjects) contiennent jusqu'à 4 Go de données non structurées.
- Stockage possible en ligne (in-line) ou hors ligne (out-of-line).
- Un LOB contient un **locator** (pointeur) vers l'objet.

Types :

- BFILE : objet binaire stocké **hors base** (lecture seule, accessible par pointeur).
- BLOB : données binaires (images, vidéos), **dans la base**, participent aux transactions.
- CLOB : texte avec encodage normal.
- NCLOB : texte en jeu de caractères national.
- Manipulables via le package DBMS_LOB.

Structure d'un bloc PL/SQL

Un bloc PL/SQL est structuré comme suit :

```
sql Copy Download  
  
[DECLARE  
  -- Variables (optionnel)]  
BEGIN  
  -- Instructions SQL + logique (obligatoire)  
[EXCEPTION  
  -- Gestion d'erreurs (optionnel)]  
END;
```

Déclaration de variables

**Nom_Variable [CONSTANT] TYPE [NOT NULL]
[[DEFAULT | :=] VALEUR] ;**

- **Nom_Variable** : doit être unique dans le bloc. Il désigne la variable.
- **TYPE** : correspond au type de la variable, qui peut être un type **scalaire** (ex. : NUMBER, VARCHAR2) ou **composé** (ex. : RECORD).
- **CONSTANT** : rend la variable **constante**, sa valeur ne pourra pas être modifiée après initialisation.
- **NOT NULL** : la variable doit obligatoirement être **initialisée** à la déclaration, sinon une **erreur** sera levée.
- **DEFAULT VALEUR** ou **:= VALEUR** : permet de donner une **valeur initiale** à la variable.

Types composites ou composés

Les types composites regroupent plusieurs champs. Ils incluent :

- **RECORD** : structure regroupant plusieurs champs de types différents.
 - Déclaration :

```
plsql

TYPE TypeEmploye IS RECORD (
    nom VARCHAR2(20),
    salaire NUMBER
);
employe TypeEmploye;
```

- **%TYPE** : permet de déclarer une variable avec le type d'une colonne ou d'une autre variable.
 - Avantages : pas besoin de connaître le type exact ; adaptation automatique si le type change.
- **%ROWTYPE** : permet de déclarer une variable représentant **toutes les colonnes d'une table**.
 - Exemple : employe employees%ROWTYPE;

⚠ Attention aux conflits de noms entre colonnes de table et variables locales.

Conflit de noms : qu'est-ce que ça veut dire ?

Quand **une variable déclarée dans un bloc PL/SQL a le même nom qu'une colonne d'une table** dans une requête SQL, **Oracle ne sait pas clairement laquelle tu veux utiliser**.

➡ Du coup, ça peut provoquer un comportement **inattendu**, voire **dangereux** (comme supprimer toutes les lignes d'une table par erreur !).

Utiliser des labels si besoin :

```
<<Main>>
DECLARE
    nom employees.nom%TYPE := 'CURTIS';
BEGIN
    DELETE FROM employees WHERE Main.nom = nom;
END;
```

Définition de sous-types

- Un sous-type restreint un type existant (plage de valeurs, contraintes).
- PL/SQL fournit des sous-types prédéfinis :
 - CHARACTER, INTEGER, PLS_INTEGER, BINARY_INTEGER
 - NATURAL, POSITIVE, SIGNTYPE, DECIMAL, FLOAT, etc.
- Sous-types avec contrainte NOT NULL :
 - SIMPLE_INTEGER, SIMPLE_FLOAT, SIMPLE_DOUBLE
- Il est aussi possible de créer ses propres sous-types :

```
SUBTYPE DateNaissance IS DATE NOT NULL;  
SUBTYPE Montant IS NUMBER(5,2);
```

Conversions de type

- Conversion **explicite** : CAST(expression AS type)
- Fonctions de conversion **implicite**:
 - TO_CHAR, TO_NUMBER, TO_DATE

Types REF

- Les types REF permettent de stocker des **références vers des objets**.
- Utilisés dans les bases objet-relationnelles (pas approfondi dans ce cours).

Visibilité des variables

- Une variable est visible dans le bloc **où elle est déclarée** et dans ses **sous-blocs**.
- Si une variable du même nom est redéclarée, elle masque l'ancienne (shadowing).

Opérateurs et expressions

- Une expression combine des opérateurs (+, -, *, AND, ||, etc.) et des opérandes (variables, constantes...).
- Respect de la **préséance des opérateurs**.
- Les expressions sont utilisées dans les affectations, les conditions, etc.

La logique trivalente et la valeur NULL

- En PL/SQL, on utilise une logique à **trois valeurs** :
 - TRUE, FALSE, NULL (inconnu)

Fonctions utiles :

- NVL(expr1, expr2) : retourne expr2 si expr1 est NULL.
- NULLIF(expr1, expr2) : retourne NULL si expr1 = expr2.
- NVL2(expr1, expr2, expr3) :
 - retourne expr2 si expr1 n'est pas NULL
 - retourne expr3 sinon.

Les séquences et les pseudo-colonnes du PL/SQL

- **Pseudo-colonnes** (utilisables en SQL) :
 - ROWID, UROWID : identifiant de ligne.
 - LEVEL : niveau hiérarchique.
 - ROWNUM : numéro temporaire d'une ligne (assigné avant ORDER BY).
 - CURRVAL, NEXTVAL : valeurs actuelle/suivante d'une séquence.

Exemple : SELECT ROWNUM, nom FROM clients;

Chapitre 3 – Les structures de contrôle

1. ♦ Introduction

- Les structures de contrôle permettent d'ajouter de la logique (conditions, boucles) dans les blocs PL/SQL.

2. ♦ Structures conditionnelles

✅ `IF THEN` : - Exécute un bloc *seulement si* la condition est vraie.

```
IF condition THEN
-- instructions
END IF;
```

✅ `IF THEN ELSE` - Gère une alternative.

```
IF condition THEN
-- instructions si vrai
ELSE
-- instructions sinon
END IF;
```

✅ `IF THEN ELSIF` - Permet plusieurs conditions.

```
IF condition1 THEN
-- bloc1
ELSIF condition2 THEN
-- bloc2
ELSE
-- bloc par défaut
END IF;
```

✅ `CASE` (Format 1) - Teste une **expression**.

```
CASE expression
WHEN valeur1 THEN ...
WHEN valeur2 THEN ...
ELSE ...
END CASE;
```

✅ `CASE` (Format 2) - Teste des **conditions**.

```
CASE
WHEN condition1 THEN ...
WHEN condition2 THEN ...
ELSE ...
END CASE;
```

 Remarque : En PL/SQL, un booléen non initialisé (NULL) donne un résultat inattendu → bien tester les 3 cas : TRUE, FALSE, UNKNOWN.

♦ Structures itératives (boucles)

 `LOOP` - Boucle infinie sans condition → ON NE L'UTILISE PAS

```
LOOP
  -- instructions
  EXIT WHEN condition;
END LOOP;
```

 `WHILE` - Répète tant que la condition est vraie.

```
WHILE condition LOOP
  -- instructions
END LOOP;
```

 `FOR` - Répète pour un intervalle déterminé (ex. 1 à 5).

```
FOR i IN 1..5 LOOP
  -- instructions
END LOOP;
```

⚠ La variable de boucle est **implicite** et **non modifiable** !

Chapitre 4 – Les exceptions

♦ 1. Utilisation des exceptions

► Déclaration

```
Nom_de_l_exception EXCEPTION;
```

Permet de signaler une erreur spécifique dans un bloc PL/SQL.

► Le gestionnaire d'exceptions

- Mot-clé : `EXCEPTION`
- Se place dans un bloc `BEGIN ... END`
- Deux types de lancement :
 - Explicitement : via `RAISE`
 - Implicitement : via Oracle (ex. division par zéro)

► La clause `WHEN OTHERS`

- Intercepte toutes les exceptions non traitées :

```
EXCEPTION
  WHEN OTHERS THEN
    DBMS_OUTPUT.PUT_LINE('Code Erreur : ' || SQLCODE || ' Message : ' || SQLERRM);
```

- ⚠ Si exécuté **côté serveur**, le message ne s'affiche pas côté client.
Pour cela `RAISE_APPLICATION_ERROR(code, message);`

◆ 2. Catégories d'exceptions

► Exceptions définies non redirigées

- Déclarées par le programmeur
- Lancées avec `RAISE`
- Pas liées à des erreurs Oracle

```
ExcNomNULL EXCEPTION;  
...  
IF LeNom IS NULL THEN RAISE ExcNomNULL;
```

► Exceptions prédéfinies

- Gérées par Oracle automatiquement

Exception	ORA	SQLCODE
ZERO_DIVIDE	ORA-01476	-1476
NO_DATA_FOUND	ORA-01403	+100
TOO_MANY_ROWS	ORA-01422	-1422
INVALID_NUMBER	ORA-01722	-1722
DUP_VAL_ON_INDEX	ORA-00001	-1
VALUE_ERROR	ORA-06502	-6502

- Exemples :
- Pas besoin de les déclarer.

► Exceptions définies redirigées

- Déclarées par le programmeur
- Reliées à des codes Oracle via :

```
PRAGMA EXCEPTION_INIT(exception_name, code_erreur);
```

- Exemple (clé étrangère invalide) :

```
ExcCleEtrangere EXCEPTION;  
PRAGMA EXCEPTION_INIT(ExcCleEtrangere, -2291);
```

► Mécanisme de propagation

- Une exception non traitée dans un bloc est remontée :
 - Au bloc englobant
 - Puis jusqu'à l'environnement d'exécution

◆ 3. Petit guide de bonnes pratiques

✓ À faire systématiquement

- Toujours gérer les exceptions possibles.
- Toujours inclure `WHEN OTHERS`.

✓ Quand les erreurs apparaissent

- Opérations : `SELECT`, `INSERT`, `UPDATE`, `DELETE`
- Conversions de types
- Chaînes de caractères

✓ Conseils

- Utiliser `%TYPE` et `%ROWTYPE` pour la robustesse
- Décider entre `COMMIT` ou `ROLLBACK` selon le contexte

✓ Cas particuliers

- Gérer les mêmes erreurs différemment en blocs imbriqués
- Identifier précisément des erreurs ayant le **même code SQL** (via `SQLERRM` + `LIKE`)
- Exemple : `IF UPPER(SQLERRM) LIKE '%CKNOM%' THEN ...`

Chapitre 5 – Les collections

🌸 Introduction aux collections

- Une **collection** est un ensemble d'éléments de **même type**, identifiés par un **indice**.
- Types de collections en PL/SQL :
 1. **Tableaux associatifs (associative arrays ou index-by tables)**
 2. **Tables imbriquées (nested tables)**
 3. **Tableaux prédimensionnés (VARARRAYs)**

🧩 1. Définir des types de collections

➤ Tableaux associatifs

- Déclaration :

sql

📄 Copier

✎ Modifier

```
TYPE nom_type IS TABLE OF type_element INDEX BY BINARY_INTEGER|PLS_INTEGER|VARCHAR2;
```

➤ Exemples d'utilisation

- Initialisation avec des constantes en tant qu'indices (positifs ou négatifs).
- Boucle pour remplir avec `SELECT * FROM ... LOOP` ou en un seul coup avec `BULK COLLECT INTO`.

➤ Tableaux avec index VARCHAR2

TYPE TypePanne IS TABLE OF INTEGER INDEX BY VARCHAR2(256);

🔗 2. Méthodes associées aux collections

Méthode	Description
EXISTS(i)	Vérifie l'existence de l'indice i.
COUNT	Nombre d'éléments dans la collection.
LIMIT	Taille maximale d'un VARRAY.
FIRST, LAST	Premier / dernier indice présent.
PRIOR(i), NEXT(i)	Indice précédent / suivant de i.
EXTEND	Ajoute un ou plusieurs éléments (sauf PL/SQL TABLE).
TRIM	Supprime le(s) dernier(s) élément(s).
DELETE	Supprime des éléments (tous ou par plage).

⚠ Certaines méthodes ne s'appliquent pas aux tableaux associatifs (ex : EXTEND, TRIM).

⚠ 3. Exceptions liées aux collections

Exception	Cause
COLLECTION_IS_NULL	Utilisation d'une collection non initialisée.
NO_DATA_FOUND	Accès à un indice inexistant.
SUBSCRIPT_BEYOND_COUNT	Indice > nombre d'éléments.
SUBSCRIPT_OUTSIDE_LIMIT	Indice hors des limites autorisées.
VALUE_ERROR	Indice null ou incorrect.

■ Chapitre 6. Des records aux collections bulk — Plan

1. Déclarations

- Utilisation de RECORD pour structurer des données hétérogènes (similaire à C).
- Exemple de définition :

```
TYPE TypeDept IS RECORD (  
    NumDep VARCHAR2(6),  
    NomDep VARCHAR2(40),  
    NumSecu VARCHAR2(8)  
);
```

- On peut aussi utiliser %TYPE ou %ROWTYPE.

2. Initialisation et accès aux champs

- Syntaxe : record.champ
- Exemple : `UnDepartement.NumDep := 99;`

3. Comparaison de 2 records

- Impossible de faire record1 = record2 directement.
- Il faut comparer champ par champ.
- Exemple de fonction possible : utiliser COALESCE pour gérer NULL.

4. Utilisation dans les opérations SQL

- Oracle 9iR2 permet :
 - INSERT, UPDATE, DELETE avec un record entier.
 - Avantage : code plus lisible.
- Exemple : `INSERT INTO Departements VALUES UnDepartement;`
- Clause RETURNING INTO pour récupérer les valeurs modifiées.

5. FORALL et collection bulk

- FORALL permet d'appliquer des opérations DML (insert/update/delete) sur une **collection**.
- Très utile pour les traitements massifs.

6. FORALL et ROLLBACK

- FORALL ne permet pas de ROLLBACK partiel par défaut.
- Il faut une gestion explicite des erreurs (cf. section 7).

7. Gestion des exceptions dans FORALL

- On utilise SAVE EXCEPTIONS et SQL%BULK_EXCEPTIONS.

8. BULK collections

- Chargement massif via BULK COLLECT.
- Exemple : `SELECT * BULK COLLECT INTO ma_table FROM Employes;`

9. RETURNING ... BULK COLLECT INTO

- Utilisation combinée avec UPDATE ou DELETE :
`UPDATE ... RETURNING ... BULK COLLECT INTO collection;`

10. FORALL + BULK COLLECT

- Très performant pour charger et modifier en masse.
- Exemple complet : charger une collection, puis faire FORALL pour traiter.

Chapitre 7 – Les procédures et les fonctions

1. CREATE PROCEDURE

```
CREATE PROCEDURE NomDeLaProcedure [(...)] IS
    [déclarations]
BEGIN
    ...
EXCEPTION
    ...
END;
```

- Paramètres : nom, mode (IN, OUT, IN OUT), type, valeur par défaut éventuelle.

⚠ Ne pas faire d'affichage depuis une procédure côté serveur en production.

2. CREATE FUNCTION

```
CREATE FUNCTION NomDeLaFonction (...)
RETURN return_type IS
    [déclarations]
BEGIN
    ...
    RETURN ...;
EXCEPTION
    ...
    RETURN ...;
END;
```

3. Paramètres avec NOCOPY

- Par défaut, les paramètres OUT et IN OUT sont passés par **valeur** (copie).
- NOCOPY permet un passage **par référence** → améliore les performances.
- ⚠ En cas d'**exception**, les données **peuvent être modifiées** malgré l'erreur.

4. RAISE_APPLICATION_ERROR

RAISE_APPLICATION_ERROR(-20000, 'Message');

- Code d'erreur de **-20000 à -20999**
- Permet de lever une erreur personnalisée

5. Notations nommées

Permettent de :

- **utiliser des valeurs par défaut**
- **fournir les arguments dans n'importe quel ordre**

Search_Client (p_NomClient => 'Dupont', p_AdresseClient => 'Namur');

6. Paramètres par défaut

- Affectables uniquement aux **paramètres IN**
- Doivent être placés **à la fin**

```
CREATE OR REPLACE PROCEDURE Select_Client
  (Nom IN CHAR, CodePostal IN NUMBER := 4500) IS
BEGIN
  ...
END;
```

Appels valides :

```
Select_Client('Dupont');
Select_Client(CodePostal => 4500, Nom => 'Dubois');
```

7. Procédures autonomes

- Ne doivent pas inclure COMMIT/ROLLBACK sauf si autonomes
- Utiliser : PRAGMA AUTONOMOUS_TRANSACTION;

Exemple :

```
CREATE OR REPLACE PROCEDURE Debug (Message IN VARCHAR2) IS
  PRAGMA AUTONOMOUS_TRANSACTION;
BEGIN
  INSERT INTO Trace (Ligne) VALUES (Message);
  COMMIT;
END;
```

Chapitre 8 – Les packages

1. Le concept du package

- Un **package** se compose de deux parties : **spécification** (éléments publics) et **corps** (implémentation et éléments privés).
- La **spécification** contient les **prototypes** des procédures/fonctions et les **variables publiques**.
- Le **corps** contient l'**implémentation**, des éléments **privés** et une **section d'initialisation** exécutée **une seule fois par session**.
- **Accès contrôlé** via le **privilège EXECUTE**, pas d'accès direct aux données.
- Avantage : **réutilisabilité** et **indépendance** du code.

2. Persistance de la session

- Une **variable publique** d'un package persiste durant toute la session Oracle.
- Exemple :

```
CREATE OR REPLACE PACKAGE PublicVarPersistante IS
    vPublicVar CHAR := 'Y';
END PublicVarPersistante;
```

Un utilisateur exécute :

```
BEGIN
    DBMS_OUTPUT.PUT_LINE ('Valeur : ' || INPRES.PublicVarPersistante.vPublicVar);
END;
```

Pour les **variables privées**, accès via des procédures :

```
CREATE OR REPLACE PACKAGE Session_Persistante IS
    PROCEDURE SetVariable (P_Valeur CHAR);
    FUNCTION GetVariable RETURN CHAR;
END SessionPersistante;
```

```
sql Copier Modifier

CREATE OR REPLACE PACKAGE BODY Session_Persistante IS
    V_PrivateVar CHAR;
    PROCEDURE SetVariable (P_Valeur CHAR) IS
    BEGIN
        V_PrivateVar := P_Valeur;
    END;

    FUNCTION GetVariable RETURN CHAR IS
    BEGIN
        RETURN V_PrivateVar;
    END;
END;
```

Chaque **session** Oracle a sa propre instance du package.

3. Principaux avantages des packages

- Réduction du **nombre d'objets stockés**.
- Séparation interface / implémentation → **moins de recompilations**.
- **Deux règles** pour éviter les recompilations :
 1. Séparer ce qui change rarement / souvent.
 2. Utiliser des **fonctions publiques** plutôt que des **constantes** dans la spécification.

4. Procédures et fonctions polymorphes et surchargées

- Surcharge possible de fonctions avec le **même nom** mais **paramètres différents**.
- Exemple :

```
CREATE OR REPLACE PACKAGE Pkgclients IS
    FUNCTION RechercheClient (p_refClient IN clients.refClient%TYPE) RETURN Clients%ROWTYPE;
    FUNCTION RechercheClient (p_nomClient IN clients.nomClient%TYPE) RETURN Clients%ROWTYPE;
END PkgClients;
```

sql

 Copier

 Modifier

```
CREATE OR REPLACE PACKAGE BODY Pkgclients IS
    FUNCTION RechercheClient (p_refClient clients.refClient%TYPE) RETURN Clients%ROWTYPE IS
        v_Client Clients%ROWTYPE;
    BEGIN
        SELECT * INTO v_Client FROM Clients WHERE RefClient = p_refClient;
        RETURN v_Client;
    END;

    FUNCTION RechercheClient (p_nomClient clients.nomClient%TYPE) RETURN Clients%ROWTYPE IS
        v_Client Clients%ROWTYPE;
    BEGIN
        SELECT * INTO v_Client FROM Clients WHERE nomClient = p_nomClient;
        RETURN v_Client;
    END;
END PkgClients;
```

5. Fonction autonome appelée à partir de SQL (je crois qu'on le voit en complément de base de données)

Une fonction autonome (ou « fonction callable depuis SQL ») est une fonction PL/SQL que tu peux utiliser directement dans une requête SQL SELECT, comme si c'était une fonction intégrée d'Oracle (ex : TO_CHAR(), SUM(), etc.).

- Calculer un **crédit client** basé sur la ristourne et le total des factures des 6 derniers mois :

```
CREATE OR REPLACE PACKAGE calcul IS
    FUNCTION credit_client(p_refclient IN clients.refclient%TYPE) RETURN number;
    PRAGMA RESTRICT_REFERENCES(credit_client,WNDS);
END calcul;
```

```
sql Copier Modifier

CREATE OR REPLACE PACKAGE BODY calcul IS
    FUNCTION credit_client(p_refclient IN clients.refclient%TYPE) RETURN number IS
        v_credit number;
    BEGIN
        SELECT sum(credit) * 10/100 INTO v_credit
        FROM (
            SELECT ristourne AS credit FROM clients WHERE refclient = p_refclient
            UNION
            SELECT total AS credit FROM factures WHERE refclient = p_refclient
            AND MONTHS_BETWEEN(CURRENT_DATE, datefacture) <= 6
        );
        RETURN v_credit;
    END;
END calcul;
```

Exemple d'utilisation SQL :

```
SELECT RefClient, NomClient, Calcul.Credit_Client(RefClient)
FROM Clients;
```

- Règles d'intégration en SQL :
 - Pas de lecture/écriture sur les mêmes tables
 - Pas de COMMIT/ROLLBACK
 - Utiliser PRAGMA RESTRICT_REFERENCES :
 - RNDS, RNPS, WNDS, WNPS, TRUST

Chapitre 10 – Les déclencheurs (Triggers)

1. Introduction

- Définition : ensemble d'actions exécutées automatiquement par le SGBD.
- Objectif : intégrer la logique métier dans la base de données.
- Applications : contraintes dynamiques, journalisation, redondance, sécurité.
- Depuis Oracle8i : déclencheurs sur LDD et événements système.

2. Déclencheurs réagissant aux instructions LMD

- **Syntaxe générale :**

```
CREATE OR REPLACE TRIGGER nom_déclencheur
  BEFORE | AFTER
  DELETE | INSERT | UPDATE [OF liste_colonne]
  ON nom_table | nom_vue
  [FOR EACH ROW]
  [WHEN condition]
  [bloc PL/SQL];
```

- **Types :**
 - Niveau **table** (sans FOR EACH ROW)
 - Niveau **ligne** (FOR EACH ROW)
- **Clé WHEN :** exécution conditionnelle.

Restrictions

- Pas de COMMIT, ROLLBACK, ni commandes DDL dans le bloc.
- Pas d'accès à une **table mutante** dans un déclencheur de ligne.
- Pas de lecture/modif des colonnes clés d'une table **contraignante**.

3. Gestion des déclencheurs

- **Création :** ENABLE ou DISABLE
- **Modification :**

SQL	Commentaires
ALTER TRIGGER NomDecl COMPILE;	Recompilation d'un déclencheur
ALTER TRIGGER NomDecl DISABLE;	Désactivation d'un déclencheur
ALTER TABLE NomTable DISABLE ALL TRIGGERS;	Désactivation de tous les déclencheurs d'une table
ALTER TRIGGER NomDecl ENABLE;	Réactivation d'un déclencheur
ALTER TABLE NomTable ENABLE ALL TRIGGERS;	Réactivation de tous les déclencheurs d'une table
DROP TRIGGER NomDecl;	Suppression d'un déclencheur

4. Ordre d'exécution

- **Ordre :**
 1. Triggers table BEFORE
 2. Analyse des lignes
 3. Triggers ligne BEFORE
 4. Modification + vérif contraintes
 5. Triggers ligne AFTER
 6. Triggers table AFTER
- Possibilité de forcer l'ordre avec FOLLOWS.